

Programming Assignment 1: FOCC and BOCC

Aayush Kumar - CO21BTECH11002

Contents

1	Introduction	2
2	BOCC Implementation	2
2.1	Main Components Overview	2
2.2	bocc-main.cpp	2
2.2.1	Global Variables and Initial Setup	2
2.2.2	updtMem() Function	2
2.2.3	Main Function	2
2.3	bocc.h	3
2.4	bocc.cpp	3
2.4.1	Transaction Initialization	3
2.4.2	Read and Write Operations	3
2.4.3	tryCommit() Function	3
2.4.4	Garbage Collection	3
2.5	Logging and Debugging	3
3	FOCC-CTA Implementation	4
3.1	Main Components Overview	4
3.2	focc-cta-main.cpp	4
3.2.1	Global Variables and Initial Setup	4
3.2.2	updtMem() Function	4
3.2.3	Main Function	4
3.3	focc-cta.h	4
3.4	focc-cta.cpp	5
3.4.1	Transaction Initialization	5
3.4.2	Read and Write Operations	5
3.4.3	tryCommit() Function	5
3.5	Logging and Debugging	5
4	Experiments	5
4.1	Experiment 1: Varying Number of Transactions	5
4.1.1	Observations	6
4.2	Experiment 2: Varying Database Size	6
4.2.1	Observations	7
4.3	Experiment 3: Varying Number of Threads	7
4.3.1	Observations	7
5	Conclusion	8

1 Introduction

The following document explains the implementation of two concurrency control mechanisms:

- **Backward Oriented Concurrency Control (BOCC)**: Validates transactions against the write sets of past committed transactions to ensure serializability.
- **Forward Oriented Concurrency Control (FOCC)**: Validates transactions by ensuring that the transaction's write set does not conflict with the read sets of other concurrent transactions.

Both implementations are described in detail, covering key components, transaction flow, and validation mechanisms.

2 BOCC Implementation

2.1 Main Components Overview

The BOCC implementation consists of three main files:

- `bocc-main.cpp`: Contains the main driver code and transaction processing logic.
- `bocc.h`: Declares key classes and functions for database management, transaction management, and concurrency control.
- `bocc.cpp`: Implements the declared functions, handling core BOCC logic.

2.2 `bocc-main.cpp`

2.2.1 Global Variables and Initial Setup

- `n`, `m`, `totTrans`, `constVal`, `lambda`, `numIters`: These are global variables storing the number of threads, size of the database, total number of transactions to execute, constant value for update, the mean of the exponential distribution, and the number of iterations per transaction.

2.2.2 `updtMem()` Function

`updtMem(11 tid)` simulates the transaction workload by:

- Initializing an exponential distribution with mean λ .
- Starting transactions in a loop and performing random read/write operations.
- Attempting to commit after performing a random number of operations.
- Continues to retry if the transaction is aborted.

2.2.3 Main Function

- Reads parameters from `inp-params.txt`.
- Initializes the database and log file.
- Creates and joins n threads to execute `updtMem()`.

2.3 bocc.h

bocc.h contains declarations for:

- **Database Class:** Stores data, locks, write lists (2D list of transactions that wrote to a data item), and active transactions set that stores the start time of all active transactions.
- **Transaction Class:** Manages read and write sets, transaction ID, and timestamps.
- Global functions: `initDatabase()`, `begin_trans()`, `read()`, `write()`, and `tryCommit()`.

2.4 bocc.cpp

2.4.1 Transaction Initialization

- `init()` initializes a transaction, records start time, and adds the transaction to `activeTrans`.
- `begin_trans()` creates a new transaction and assigns it a unique ID.

2.4.2 Read and Write Operations

- `read()` reads a value from the database or transaction's write set.
- `write()` buffers a value in the transaction's write set for future commit.

2.4.3 tryCommit() Function

The `tryCommit()` function:

- Acquires locks on read and write sets. Locks on both sets are acquired to ensure correctness.
- Validates that the `readSet` of the transaction does not conflict with the `writeSet` of other transactions that are committed and ended after the current transaction started.
- Restarts the transaction if validation fails.
- Updates the database and `write_list` if validation succeeds.
- Removes the transaction from the set of active transactions and commits changes.
- Calls `collectGarbage()` to remove old committed transactions.

2.4.4 Garbage Collection

`collectGarbage()` removes old committed transactions that are no longer needed by comparing transaction end times with the earliest active transaction. This is done by maintaining a `activeTrans` set that stores the start time of all active transactions. The transactions that committed before the earliest active transaction are removed from the database.

2.5 Logging and Debugging

`log()` writes transaction details to `BOCC-log.txt` to facilitate debugging and performance analysis.

3 FOCC-CTA Implementation

3.1 Main Components Overview

The FOCC-CTA implementation consists of three main files:

- `focc-cta-main.cpp`: Contains the main driver code and transaction processing logic.
- `focc-cta.h`: Declares key classes and functions for database management, transaction management, and concurrency control.
- `focc-cta.cpp`: Implements the declared functions, handling core FOCC logic.

3.2 focc-cta-main.cpp

3.2.1 Global Variables and Initial Setup

- `n`, `m`, `totTrans`, `constVal`, `lambda`, `numIters`: These are global variables storing the number of threads, size of the database, total number of transactions to execute, constant value for update, the mean of the exponential distribution, and the number of iterations per transaction.

3.2.2 updtMem() Function

`updtMem(11 tid)` simulates the transaction workload by:

- Initializing an exponential distribution with mean λ .
- Starting transactions in a loop and performing random read/write operations.
- Attempting to commit after performing a random number of operations.
- Continues to retry if the transaction is aborted.

3.2.3 Main Function

- Reads the input parameters from `inp-params.txt`.
- Initializes the database using `initDatabase()`.
- Opens a log file named `FOCC-log.txt` to record events.
- Creates and joins n threads that execute `updtMem()`.

3.3 focc-cta.h

`focc.h` contains declarations for:

- **Database Class**: Stores data, locks, and read lists (2D list of transactions that read a data item).
- **Transaction Class**: Manages read and write sets, transaction ID, and timestamps.
- Global functions: `initDatabase()`, `begin_trans()`, `read()`, `write()`, and `tryCommit()`.

3.4 focc-cta.cpp

3.4.1 Transaction Initialization

- `init()` initializes a transaction, records start time, and resets read and write sets.
- `begin_trans()` creates a new transaction and assigns it a unique ID.

3.4.2 Read and Write Operations

- `read()` reads a value from the database or transaction's write set.
- `write()` buffers a value in the transaction's write set for future commit.

3.4.3 tryCommit() Function

The `tryCommit()` function:

- Acquires locks on read and write sets.
- Validates that the intersection between the transaction's `writeSet` and the `readSet` of other transactions is null.
- Restarts the transaction if validation fails.
- Updates the database if validation succeeds.
- Removes the transaction's ID from the `read_list`.

3.5 Logging and Debugging

`log()` writes transaction details to `F0CC-log.txt` to facilitate debugging and performance analysis.

4 Experiments

The experiments were performed on **11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz** with **8GB RAM** and **8 cores**. The following parameters remain constant in all experiments:

- **Constant Value** (*constVal*): 1000
- **Lambda** (λ): 20
- **Number of iterations per transaction** (*numIters*): 20

4.1 Experiment 1: Varying Number of Transactions

We fix the number of threads (n) at 8, database size (m) at 1000, and vary the total number of transactions (*numTrans*) from 1000 to 5000 and measure the average time taken by a transaction to commit and average number of aborts per transaction. The value of time is in nanoseconds. The results are shown in Table 1 and Figure 1.

Transactions	BOCC Time	FOCC-CTA Time	BOCC Aborts	FOCC Aborts
1000	1009826029	1561619977	1.494	2.847
2000	1040492200	1593802616	1.572	2.947
3000	1041854860	1635373294	1.576	3.046
4000	1044733395	1617138626	1.583	3.129
5000	1042775006	1621911401	1.582	3.003

Table 1: Execution Time for Varying Number of Transactions

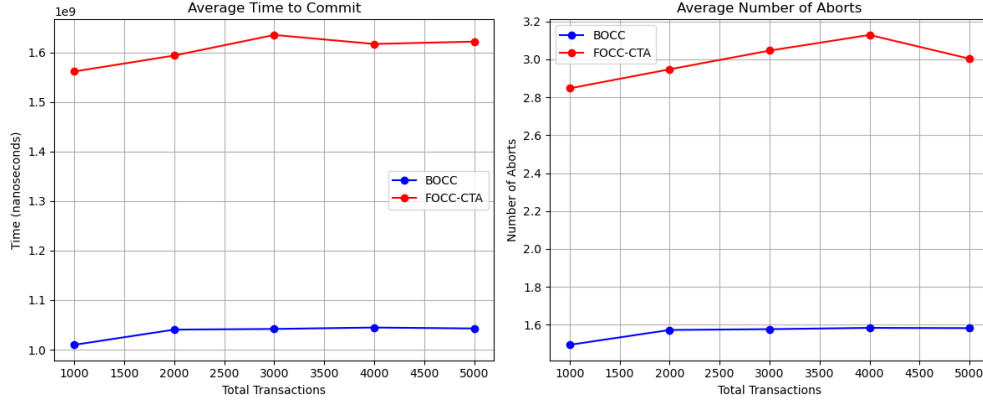


Figure 1: Execution Time and Aborts for Varying Number of Transactions

4.1.1 Observations

- The average time taken and the average abort count per transaction remains almost constant with the number of transactions for both BOCC and FOCC. This is since the performance is determined mainly by the number of concurrent transactions and not the total number of transactions. As the number of transactions increases, total time taken increases linearly but the average time per transaction remains constant.
- BOCC has a lower average time per transaction and abort rate compared to FOCC.

4.2 Experiment 2: Varying Database Size

We fix the number of threads (n) at 8, total number of transactions ($numTrans$) at 1000, and vary the database size (m) from 1000 to 5000. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 2 and Figure 2.

Transactions	BOCC Time	FOCC-CTA Time	BOCC Aborts	FOCC Aborts
1000	1524794577	7738178418	2.772	18.155
2000	1090774353	1770937883	1.701	3.383
3000	907888961	1078054165	1.245	1.672
4000	812295501	872103093	1.001	1.156
5000	747733680	739130970	0.843	0.842

Table 2: Execution Time and Aborts for Varying Number of Transactions

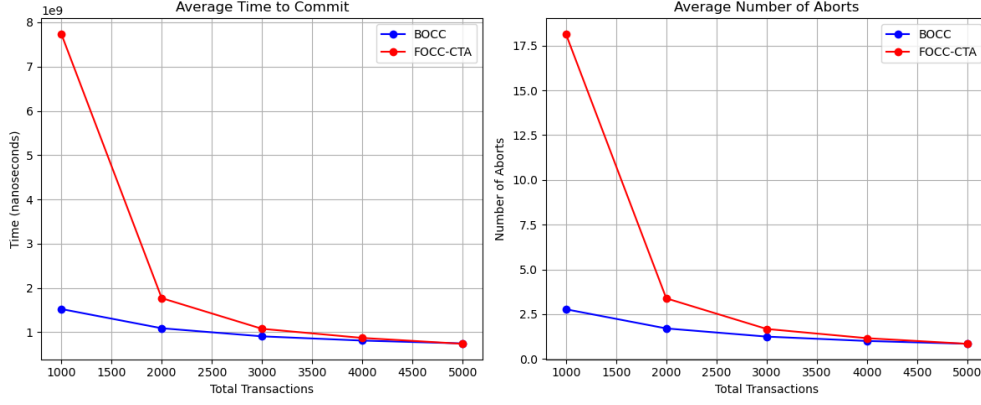


Figure 2: Execution Time and Aborts for Varying Database Size

Transactions	BOCC Time	FOCC-CTA Time	BOCC Aborts	FOCC Aborts
2	531938936	493188214	0.317	0.223
4	722717974	736257497	0.789	0.822
8	1040972608	1620280357	1.580	3.007
16	1526237603	7270318879	2.780	17.007
32	2337992689	169132640009	4.799	417.903

Table 3: Execution Time and Aborts for Varying Number of Transactions

4.2.1 Observations

- The average time taken and the average abort count per transaction decreases with the database size for both BOCC and FOCC. This is because a larger database size reduces the probability of conflicts between transactions.
- BOCC has a lower average time per transaction and abort rate compared to FOCC for smaller database sizes. For larger database sizes, the performance difference between BOCC and FOCC decreases with FOCC performing slightly better.

4.3 Experiment 3: Varying Number of Threads

We fix the database size (m) at 1000, total number of transactions ($numTrans$) at 1000, and vary the number of threads (n) from 2 to 32. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 3 and Figure 3.

4.3.1 Observations

- The average time taken and the average abort count per transaction increases with the number of threads for both BOCC and FOCC. This is because more threads lead to increased contention for resources, resulting in more conflicts and aborts.
- For very small number of threads, FOCC has a lower average time per transaction and abort rate compared to BOCC. However, as the number of threads increases, BOCC performs better than FOCC. For a large number of threads, FOCC has a significantly higher average time per transaction and abort rate compared to BOCC.

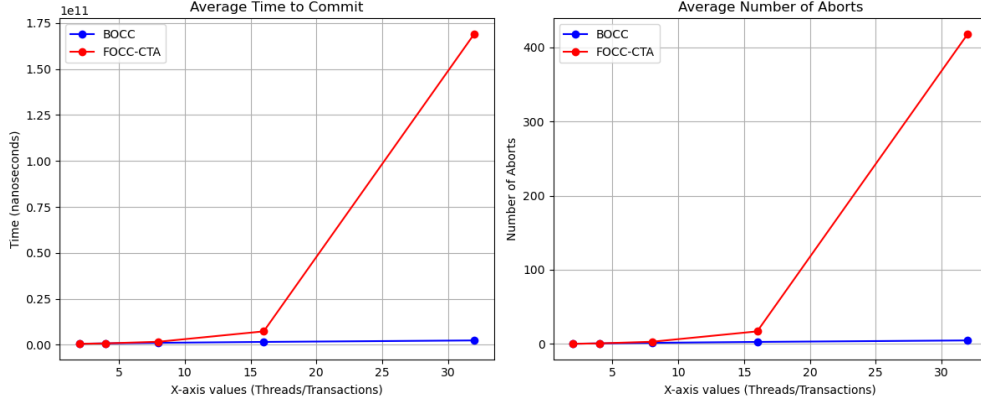


Figure 3: Execution Time and Aborts for Varying Number of Threads

5 Conclusion

The implementation and experiments demonstrate the working of BOCC and FOCC concurrency control mechanisms. The experiments show that BOCC performs better than FOCC in terms of average time per transaction and abort rate. The performance of both mechanisms is affected by the number of transactions, database size, and number of threads. BOCC is more efficient for smaller database sizes and a larger number of threads, while FOCC performs better for a smaller number of threads.

A possible reason for the better performance of BOCC is that it validates transactions against the write sets of past committed transactions, while FOCC validates transactions against the read sets of other concurrent transactions. So there are order of $O(n)$ transactions to validate in FOCC since there can be $O(n)$ transactions that are concurrent with the current transaction, while in BOCC it is less than $O(n)$ since in our experiments, all the transactions take almost the same time, so they try to commit at almost the same time. So, the first transaction that tries to commit will have to validate against no transactions, the second transaction will have to validate against 1 transaction, and so on, till the n -th transaction and then the pattern continues. Here, n is the number of threads. This results in a lower number of conflicts and hence a lower number of aborts in BOCC compared to FOCC.

However, for large database size or for small number of threads, the probability of conflicts are small and hence FOCC performs better than BOCC. This is because BOCC has a larger overhead of garbage collection and maintaining the write lists for all the committed transactions where as FOCC only maintains the read lists for all the active transactions and hence needs no garbage collection.